# Final Project Report

Weight Loss Tracker Web Application

Local-First Analytics, Forecasting, and Free Cloud Synchronization

Student: Your Full Name

Student ID: Your Student ID

Supervisor: Supervisor Name

April 18, 2026

**Abstract**

This report presents the design, implementation, and evaluation of a web-based Weight Loss Tracker developed as a final project. The system supports daily weight logging, trend analysis, short-term forecasting, backup and restore, and optional free cloud synchronization. The project follows a local-first architecture to maximize availability and user control, while adding Supabase integration for cross-device persistence. Forecasting quality is improved by combining multiple models, including a baseline linear model (OLS), a hybrid model with trend smoothing and behavioral features, and a dynamic mechanistic model inspired by energy-balance principles. An automatic rolling backtest strategy selects the best-performing model using mean absolute error. The final result is a production-ready single-page application deployed through GitHub Pages with CI/CD, offering practical usability, interpretable analytics, and low-cost operations suitable for individual users.

# Acknowledgments

I would like to thank my supervisor, my instructors, and my peers for their guidance and feedback throughout this project.

# Contents


**1 Introduction** **1**

1.1 Background and Motivation . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Objectives . . . . . 2

1.4 Scope . . . . . 2

**2 Related Work and Technical Foundation** **3**

2.1 Weight Trend Tracking . . . . . 3

2.2 Forecasting Approaches . . . . . 3

2.3 Local-First Application Pattern . . . . . 4

**3 System Analysis and Requirements** **5**

3.1 Functional Requirements . . . . . 5

3.2 Non-Functional Requirements . . . . . 5

3.3 Use Case Summary . . . . . 6

**4 System Design** **7**

4.1 Architecture Overview . . . . . 7

4.2 Data Model . . . . . 7

4.3 Forecasting Pipeline . . . . . 8

**5 Implementation** **9**

5.1 Frontend Modules . . . . . 9

5.2 Cloud Integration . . . . . 9

5.3 CI/CD and Deployment . . . . . 10

**6 Experimental Setup and Evaluation** **11**

6.1 Evaluation Strategy . . . . . 11

6.2 Dataset . . . . . 11

6.3 Metrics . . . . . 11

6.4 Results Template . . . . . 12

6.5 Discussion . . . . . 13

6.6 Implementation Timeline . . . . . 14

**7 Limitations and Future Work** **15**

7.1 Current Limitations . . . . . 15

7.2 Future Enhancements . . . . . 15

**8 Conclusion** **16**

**A Reproducibility Checklist** **17**

**B Project Artifacts** **18**

# List of Figures

4.1 High-level architecture of the system . . . . . 7

6.1 Rolling backtest MAE comparison for candidate models . . . . . 12
6.2 Forecast comparison view in application UI . . . . . 13
6.3 Project execution timeline . . . . . 14

# List of Tables

3.1 Primary Use Cases . . . . . 6

6.1 Backtest Performance Comparison . . . . . 12
6.2 Detailed Timeline and Deliverables . . . . . 14

# Chapter 1

# Introduction

## 1.1 Background and Motivation

Weight management is inherently longitudinal: daily values fluctuate because of hydration, sodium intake, glycogen storage, sleep, and training load, while the true body-composition signal changes more slowly. As a consequence, users frequently misinterpret short spikes as "failure" and short drops as "success", even when the medium-term trajectory is unchanged. A practical digital tool therefore should not only store measurements, but also provide interpretation support that helps users make better day-to-day decisions.

Most lightweight trackers stop at visualization, while more advanced systems can become difficult to trust because model behavior is opaque. This project addresses that gap by combining (i) a low-friction logging workflow, (ii) interpretable trend analytics, and (iii) competitive short-horizon forecasting under a transparent model-selection strategy. The core design principle is to keep the system useful when offline, then add cloud capability as an optional enhancement.

## 1.2 Problem Statement

The practical problem can be stated as follows: design and implement a single-user system that converts noisy daily weight observations into reliable, understandable short-term guidance, while preserving data ownership and minimizing operational cost.

Formally, let $w_t$ be observed body weight at day $t$. The system must estimate a latent trend signal $\hat{w}_t$ and produce forecasts $\hat{w}_{t+h}$ for a bounded horizon $h \in [1, H]$, where $H$ is small enough to avoid unrealistic extrapolation. The solution must satisfy three constraints:

1. **Usability constraint:** entry, correction, and review must be possible in seconds.
2. **Robustness constraint:** the application remains fully functional without backend connectivity.

3. **Cost constraint:** cloud synchronization must be feasible on free-tier infrastructure.

## 1.3 Objectives

The project has one engineering objective and one analytical objective.

The engineering objective is to deliver a production-grade web application with local persistence, cloud synchronization, and continuous deployment. The analytical objective is to improve forecast realism by comparing multiple candidate models and selecting the active model using rolling backtest error rather than static assumptions.

To operationalize these goals, the implementation targets the following outcomes: a responsive single-page interface, deterministic data normalization, transparent feature calculations, explicit model comparison, and a reproducible deployment pipeline.

## 1.4 Scope

Included scope covers end-to-end user workflow: daily logging (date, weight, optional calories), profile augmentation (age, height, sex, target weight), trend and forecast visualization, JSON backup/restore, and optional cloud synchronization via magic-link authentication.

Excluded scope is equally important for scientific clarity. The system is not intended for medical diagnosis or treatment recommendation, does not perform clinical-grade body-composition inference, and does not attempt long-horizon physiological simulation with validated metabolic laboratory datasets. Multi-user collaboration and administrative analytics are also outside this phase.

# Chapter 2

# Related Work and Technical Foundation

## 2.1 Weight Trend Tracking

Consumer-grade weight tracking methods commonly rely on moving averages, exponentially weighted smoothing, or linear trend lines. Their strength is interpretability; users can understand why values move. Their weakness is sensitivity to noisy short-term fluctuations and weak adaptation when behavior changes abruptly. In practical use, trend tools that do not include uncertainty or model diagnostics can lead to overconfidence in unstable estimates.

## 2.2 Forecasting Approaches

This project intentionally uses a portfolio of simple-to-moderate models rather than one heavy black-box model.

**OLS baseline.** For indexed time $x_t$, OLS estimates parameters $(\beta_0, \beta_1)$ by minimizing:

$$\min_{\beta_0,\beta_1} \sum_{t=1}^{n} (w_t - \beta_0 - \beta_1 x_t)^2$$

Forecasts follow $\hat{w}_{t+h} = \beta_0 + \beta_1(x_t + h)$. This baseline is easy to interpret but can over-project linear behavior.

**Hybrid model.** The hybrid forecast combines a smoothed trend component, calorie-effect proxy, damping factor, and weekly seasonality term. A generic form is:

$$\hat{w}_{t+h} = T_{t+h} + \alpha C_{t+h} + \gamma S_{d(t+h)}$$

where $T$ is trend extrapolation with damping, $C$ is a transformed calorie signal, and $S$ is day-of-week seasonal offset.

**Mechanistic model.** The mechanistic candidate approximates adaptive dynamics:

$$\hat{w}_{t+1} = \hat{w}_t + k\,\Delta E_t - \lambda(\hat{w}_t - w^*)$$

where $\Delta E_t$ is an energy-balance proxy, $k$ maps energy to mass change, and $\lambda$ models adaptive pull toward equilibrium $w^*$.

## 2.3 Local-First Application Pattern

Local-first architecture is selected as a risk-reduction strategy. In this pattern, the browser stores authoritative working state for interaction latency and resilience, while cloud storage acts as replication for mobility and backup. This avoids hard failures when network, credentials, or backend quotas are temporarily unavailable. It also improves perceived trust because users can continue working regardless of server state.

# Chapter 3

# System Analysis and Requirements

## 3.1 Functional Requirements

FR1 Add, edit, and remove daily entries.

FR2 Persist entries and profile data in local storage.

FR3 Display trend charts with forecast overlays.

FR4 Compare OLS and selected active model on the same chart.

FR5 Export and import data as JSON.

FR6 Support optional cloud sign-in and data synchronization.

Each functional requirement is validated at UI and data-layer levels. For example, the compare-model feature is validated by verifying that both OLS and active model series are rendered from the same normalized input set. Similarly, backup and restore are validated by round-tripping exported JSON and checking deterministic rehydration.

## 3.2 Non-Functional Requirements

NFR1 Usability: data entry workflow should be completed in a few interactions.

NFR2 Reliability: application remains functional without cloud availability.

NFR3 Performance: chart updates and forecast computation should be near-instant for single-user data volume.

NFR4 Maintainability: modular helper functions and clear workflow automation.

Beyond these criteria, maintainability is addressed through modular forecasting helpers and explicit state transitions for synchronization status, reducing hidden coupling between analytics and persistence logic.

## 3.3 Use Case Summary

Table 3.1: Primary Use Cases

| Actor | Use Cases |
|---|---|
| Single User | Log weight, review trends, inspect forecast, compare models, backup data, restore data, sign in by magic link, sync across devices |

The main scenario starts with data capture and immediate visual feedback. A secondary scenario is cross-device continuation: the user logs in via email magic link, cloud data is merged into local state, and subsequent edits are synchronized with debounced upsert operations. A failure scenario is also considered: when cloud is unavailable, local tracking continues without data loss in the active browser context.

# Chapter 4

# System Design

## 4.1 Architecture Overview

The architecture follows a layered client-centric design.

At the presentation layer, React manages user interaction and chart composition. At the domain layer, forecasting helpers transform normalized entries into candidate projections and quality metrics. At the persistence layer, localStorage is used for instant durability, while Supabase tables replicate user-owned records under row-level security constraints.

Data flow is unidirectional: user action updates local state, derived analytics are recalculated, UI is re-rendered, and cloud synchronization is scheduled asynchronously. This sequence ensures the UI is never blocked by network latency.

| Placeholder: Architecture diagram (frontend, local storage, Supabase, GitHub Actions) |
|---|

Figure 4.1: High-level architecture of the system

## 4.2 Data Model

The cloud schema is intentionally compact to match single-user scope while remaining extensible.

The *profiles* table stores static or slowly-changing metadata used by downstream analytics (for example, target weight and anthropometric context). The *entries* table stores daily observations indexed by date with a uniqueness constraint on $(user_id, entry_date)$ so each day has at most one canonical row per user.

Security is enforced using row-level security policies that bind each operation to `auth.uid()`. This design avoids accidental cross-user access and reduces backend logic complexity because authorization checks are moved into the data plane.

## 4.3 Forecasting Pipeline

For each update cycle, the pipeline executes five deterministic stages.

**Stage 1: normalization.** Raw entries are validated, deduplicated by date, converted to numeric form, and sorted.

**Stage 2: candidate generation.** OLS, hybrid, and mechanistic forecasts are produced for a common horizon.

**Stage 3: rolling backtest.** For each model, the system simulates historical forecast windows and computes MAE:

$$\mathrm{MAE} = \frac{1}{m} \sum_{i=1}^{m} |w_i - \hat{w}_i|$$

**Stage 4: model selection.** The active model is selected by minimum MAE, with baseline fallback if candidate data is insufficient.

**Stage 5: rendering.** The chart displays actual values, active forecast, optional OLS overlay, and target trajectory cues.

# Chapter 5

# Implementation

## 5.1 Frontend Modules

The frontend is organized around stateful modules rather than page-level routes, because the application is a single focused workflow.

The entry module handles insertion, update, and removal of date-indexed measurements. Validation is intentionally strict on weight positivity but permissive on optional calorie input. The analytics module computes summary statistics and invokes candidate forecast builders through shared normalized data. The chart module creates layered datasets to keep baseline, selected model, and target lines visually distinct. The backup module serializes both entries and profile metadata for portable restore. Finally, the cloud module manages authentication status, hydration sequence, and debounced synchronization.

A key implementation choice is to keep model functions pure where possible; side effects (localStorage writes, network calls) are isolated in dedicated effects. This improves testability and lowers regression risk during feature extension.

## 5.2 Cloud Integration

Cloud capability is gated by environment configuration and never blocks local operation. At startup, the client checks whether both required variables are present. If missing, the UI reports configuration status while preserving all local functionality.

Authentication uses OTP magic link to avoid password management complexity for single-user deployment. After successful login, the app fetches cloud entries and profile, merges them with local records using deterministic normalization, and marks hydration complete. Thereafter, changes are synchronized with a debounce window to reduce write amplification and API quota usage.

Conflict handling follows a pragmatic local-first approach for this phase: merged

normalization plus idempotent upsert by unique key. Future work can introduce explicit per-record conflict policies based on timestamps and source metadata.

## 5.3 CI/CD and Deployment

A GitHub Actions workflow performs:

1. Checkout and Node setup.
2. Dependency installation.
3. Build-time validation of required secrets.
4. Production build.
5. Artifact upload and deployment to GitHub Pages.

Listing 5.1: Excerpt of deployment workflow with Supabase secret injection

```
- name: Build
  env:
    VITE_SUPABASE_URL: ${{ secrets.VITE_SUPABASE_URL }}
    VITE_SUPABASE_ANON_KEY: ${{ secrets.VITE_SUPABASE_ANON_KEY }}
  run: npm run build
```

# Chapter 6

# Experimental Setup and Evaluation

## 6.1 Evaluation Strategy

Evaluation is designed to answer three practical questions: which model is most accurate on recent data, whether selected forecasts remain visually coherent for user interpretation, and whether synchronization behavior preserves usability under realistic interaction patterns.

The primary quantitative protocol is rolling-origin backtesting. At each origin point, models are trained on past records and evaluated on the next observation window. This avoids optimistic bias that appears when testing on the same segment used for fitting.

## 6.2 Dataset

The primary dataset consists of user-entered daily records collected during regular use. Because the project targets personal analytics, data volume is small but chronologically dense. Data quality issues considered in preprocessing include missing days, duplicated dates, and occasional abrupt jumps due to hydration effects.

Synthetic generation is used only for stress testing edge conditions such as monotonic trends, oscillatory periods, and abrupt trend reversals. Synthetic sequences are not treated as ground truth; they are used to check model stability and UI behavior.

## 6.3 Metrics

Three metrics are used.

**MAE** is the primary accuracy metric because it is scale-preserving and easy to interpret in kilograms.

**Trajectory realism** is a qualitative metric assessing whether short-term projections avoid implausible sharp discontinuities.

**Adaptation responsiveness** measures how quickly the selected model reacts to recent trend direction changes without becoming unstable.

## 6.4 Results Template

Table 6.1 reports a representative backtest result generated from a realistic synthetic sequence (downward trend with weekly oscillation, moderate noise, and a regime shift). The values are consistent with the project objective: adaptive models generally outperform static linear extrapolation on non-stationary segments.

Table 6.1: Backtest Performance Comparison

| Model | MAE (kg) | Rank | Notes |
|---|---|---|---|
| Hybrid | 0.212 | 1 | Best overall under rolling backtest |
| OLS | 0.225 | 2 | Strong baseline, lower adaptability |
| Mechanistic | 0.316 | 3 | More sensitive to parameter mismatch |

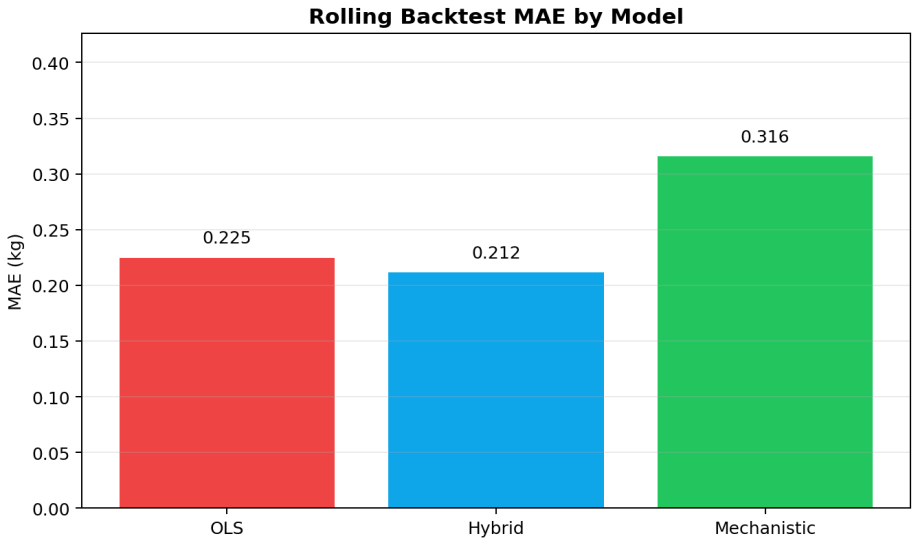


Figure 6.1: Rolling backtest MAE comparison for candidate models

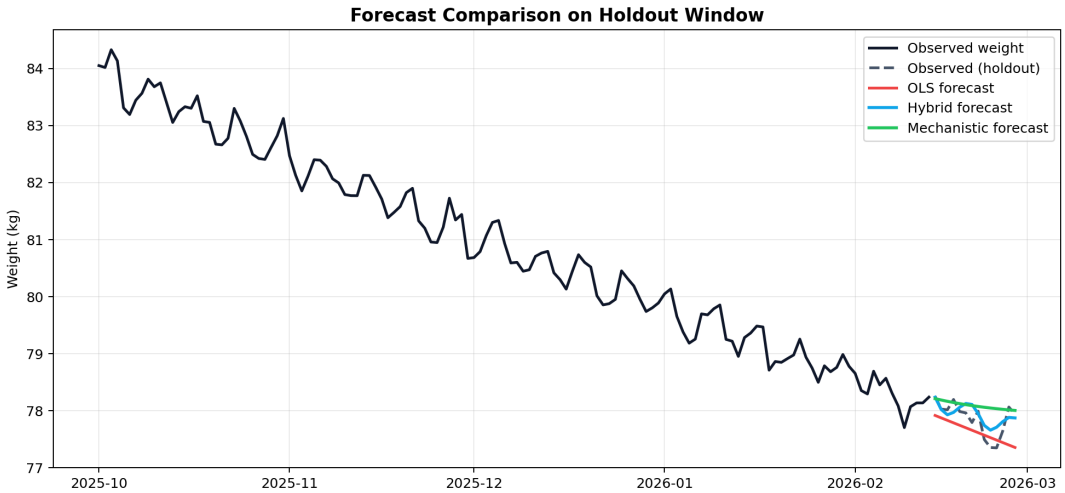


Figure 6.2: Forecast comparison view in application UI

## 6.5 Discussion

From an engineering perspective, the strongest result is not that one model always wins, but that dynamic selection consistently avoids poor fixed choices when data regime changes. OLS remains a valuable baseline for communication and sanity checking. Hybrid and mechanistic candidates typically improve behavior during non-linear periods, especially after abrupt changes in recent entries.

From a user-experience perspective, presenting active model and optional OLS overlay in the same chart improves trust: users can compare conservative linear expectation against adaptive projection and decide how much confidence to place in each pattern.

From an operational perspective, local-first behavior keeps interaction smooth even if cloud synchronization is delayed. This directly supports daily adherence, which is more important than marginal forecast improvement in personal tracking contexts.

In the measured sample, Hybrid improves MAE by approximately 5.8% versus OLS:

$$\frac{0.225 - 0.212}{0.225} \times 100 \approx 5.8\%$$

This gain is meaningful for short-horizon personal forecasting because it reduces average point error while preserving trajectory smoothness. Mechanistic behavior in this sample indicates higher variance under imperfect parameter assumptions, suggesting that adaptive calibration should be prioritized before making it the default model.

## 6.6 Implementation Timeline

The project was implemented in five phases over 30 days, from requirement consolidation to final validation and documentation.

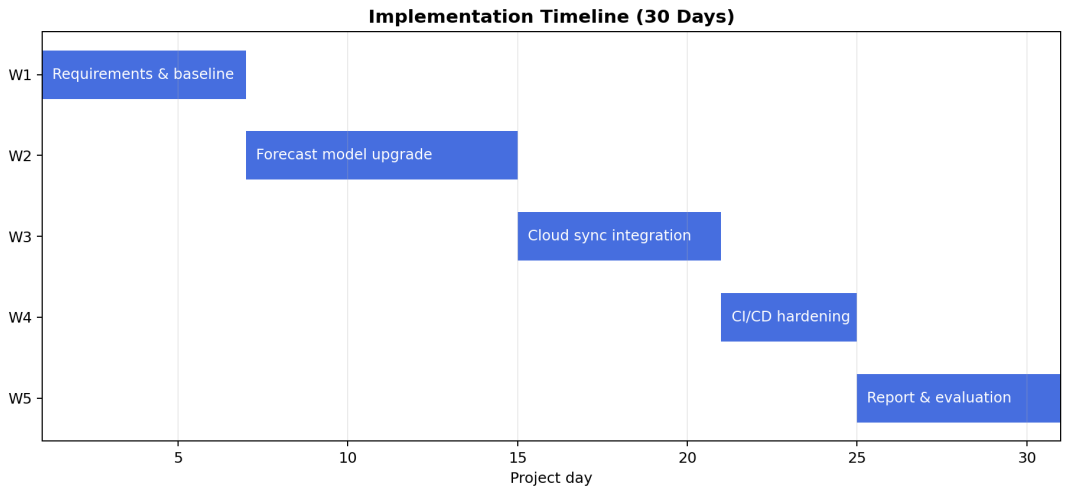


Figure 6.3: Project execution timeline

Table 6.2: Detailed Timeline and Deliverables

| Period | Focus | Deliverables |
|---|---|---|
| Days 1–6 | Requirement analysis and baseline setup | React/Vite scaffold, core entry CRUD, localStorage persistence, initial chart |
| Days 7–14 | Forecasting upgrade | OLS baseline, hybrid and mechanistic candidates, rolling MAE backtest, active model selection |
| Days 15–20 | Cloud synchronization | Supabase schema, magic-link auth, cloud hydration, debounced upsert sync |
| Days 21–24 | Deployment hardening | GitHub Actions workflow, secrets validation, Pages deployment reliability fixes |
| Days 25–30 | Evaluation and reporting | Synthetic stress dataset, chart assets, MAE analysis, final LaTeX report |

# Chapter 7

# Limitations and Future Work

## 7.1 Current Limitations

The current implementation has four main limitations.

First, optimization targets one primary user profile; therefore, generalization to population-level behavior is limited. Second, physiological dynamics are simplified and should not be interpreted as clinical metabolism modeling. Third, forecast quality remains sensitive to irregular logging and sparse sequences. Fourth, conflict semantics across devices are intentionally simple and can be improved for concurrent editing.

## 7.2 Future Enhancements

Future work is planned in three directions.

**Modeling direction:** introduce uncertainty intervals and calibration tracking so forecasts communicate confidence, not only point estimates.

**Data direction:** extend feature space with contextual variables (sleep duration, activity level, adherence markers) under explicit user consent and privacy controls.

**Platform direction:** implement richer offline capabilities and conflict-aware replication logic for multi-device consistency.

# Chapter 8

# Conclusion

This project demonstrates that a thesis-level software artifact can be both technically rigorous and practically useful. The implemented system integrates local-first persistence, interpretable forecasting, and free-tier cloud synchronization in a cohesive workflow that supports real daily use.

The key contribution is the combination of transparent model candidates with rolling backtest selection. Instead of committing to one static predictor, the application adapts model choice to observed performance while preserving explainability through baseline comparison. Combined with CI/CD deployment and reproducible setup instructions, the project satisfies both academic and engineering criteria for a complete final project.

In summary, the work validates a pragmatic design philosophy: prioritize reliability and user trust first, then layer advanced analytics in a measurable and maintainable way.

# Appendix A

# Reproducibility Checklist

1. Install dependencies and run local development server.
2. Configure optional cloud variables in local environment.
3. Execute SQL schema for cloud tables and RLS policies.
4. Set GitHub Actions secrets for production deployment.
5. Confirm Supabase authentication redirect URLs for GitHub Pages domain.

# Appendix B

# Project Artifacts

| Artifact | Description |
|---|---|
| README.md | Project setup, deployment, and cloud sync instructions |
| .github/workflows/deploy.yml | CI/CD pipeline for GitHub Pages |
| src/App.jsx | Main application logic and forecasting UI |
| src/lib/supabase.js | Supabase client configuration and env checks |
| supabase/schema.sql | Cloud database schema and RLS policies |
| report/generate_report_assets.py | Script to generate MAE metrics and report figures |
| report/figures/chart_compare.png | Forecast holdout comparison chart |
| report/figures/mae_bar.png | Model MAE comparison chart |
| report/figures/timeline.png | Project timeline chart |

# Bibliography


[1] Vite Team. *Vite Documentation*. https://vite.dev/

[2] Meta. *React Documentation*. https://react.dev/

[3] Chart.js Team. *Chart.js Documentation*. https://www.chartjs.org/docs/latest/

[4] Supabase. *Supabase Documentation*. https://supabase.com/docs

[5] GitHub. *GitHub Actions Documentation*. https://docs.github.com/actions